TrialMatch-LLM: Evidence-Grounded Criterion-Level Clinical-Trial Screening with a Fine-Tuned Small Decoder, Explicit KV Caching, Tiled Attention and Speculative Decoding

Ali Mazhar
23i-0796, BS Computer Science (BSCS)
FAST National University of Computer and Emerging Sciences, Islamabad
i230796@isb.nu.edu.pk

Abstract—We build and evaluate a complete, locally executable Transformer system that decides how individual inclusion and exclusion criteria of a clinical trial apply to a patient and cites the supporting patient sentences. Using only the TrialGPT Criterion Annotations dataset and a fixed patient-grouped 37/8/8 split, we fine-tune Qwen2.5-1.5B-Instruct with LoRA to emit a JSON object containing evidence sentence IDs followed by one of four expert label categories, including not enough information and not applicable. On the 206 held-out test annotations, fine-tuning raises accuracy from 0.228 to 0.694, macro-F1 from 0.192 to 0.386 and evidence micro-F1 from 0.236 to 0.546, while rare classes (not included, exclusion-side not enough information) remain unsolved. We additionally implement, from scratch, a Qwen2 forward pass with an explicit key/value cache that is token-identical to an uncached reference at all nine tested input/output lengths and decodes 14.2× faster at 1,024 input tokens; a tiled online-softmax attention kernel (maximum error 2.0×10^{-6} vs. fp64 in fp32); and greedy speculative decoding with a 0.5B draft, which is token-identical to greedy decoding with up to 96% draft acceptance but 0.61–0.92× the speed of plain decoding on a T4, a slow-down we explain by layer-count-bound latency. All components are exposed in a browser application that screens unseen notes and exports a provenance-tagged report.

Index Terms—clinical trial matching, large language models, LoRA, KV cache, FlashAttention, speculative decoding, evidence extraction

I. Introduction

Recruitment is a major bottleneck of clinical trials, and checking a patient against dozens of free-text eligibility criteria is laborious. TrialGPT [1] showed that large proprietary models can predict criterion-level eligibility and cite evidence. Our goal is a system that (i) runs entirely on free hardware with open weights, (ii) preserves the full label space, including insufficient-information and not-applicable outcomes rather than forcing a yes/no decision, (iii) grounds every decision in patient sentence IDs, and (iv) makes the inference mechanisms that dominate cost (KV caching, attention, decoding) explicit and verified.

Problem statement. Given a patient note split into numbered sentences $S = \{s_0, \ldots, s_{n-1}\}$, a criterion type $t \in \{\text{inclusion}, \text{exclusion}\}$ and criterion text c, predict a label $y \in \mathcal{Y}_t$ and an evidence set $E \subseteq \{0, \ldots, n-1\}$, where $\mathcal{Y}_{\text{inc}} = \{$included, not included, not enough information, not applicable$\}$ and $\mathcal{Y}_{\text{exc}} = \{$excluded, not excluded, not enough information, not applicable$\}$.

Contributions. (1) A leakage-controlled fine-tuning and evaluation pipeline with the mandated split; (2) a from-scratch cache-aware Qwen2 decoder with prefix caching; (3) a tiled online-softmax attention kernel; (4) a greedy speculative decoder with explicit cache rollback; (5) a browser application with live benchmarks; (6) an honest analysis including failures and slow-downs.

II. Related Work

Patient–trial matching. The patients in our data originate from the SIGIR 2016 test collection [2]. TrialGPT [1] prompts GPT-4 to produce criterion-level labels, explanations and evidence sentences, then aggregates them into trial rankings. It relies on a proprietary API; we instead study what a 1.5B open model can learn from the expert annotations, and we never use the GPT-4 predictions stored in the dataset.

Transformers and efficient adaptation. We build on the decoder-only Transformer [3] with rotary position embeddings [4] and grouped-query attention (GQA) [5], as used by Qwen2.5 [6] and Llama 3 [7]. LoRA [8] trains low-rank updates $\Delta W = \frac{\alpha}{r} BA$ while freezing W; QLoRA [9] additionally quantises W to 4-bit NF4. Full fine-tuning of a 1.5B model needs about 4× the weight memory for Adam states and would over-fit 602 examples; LoRA trains ≈1.2% of the parameters. QLoRA reduces frozen-weight memory about 4× at the cost of de-quantisation overhead and quantisation error; on a 16 GB T4 the 1.5B model fits without it (§V-G).

Efficient inference. Pope et al. [10] analyse KV-cache memory and why autoregressive decoding is memory-bandwidth bound; PagedAttention [11] manages cache memory in blocks. FlashAttention [12] computes exact attention in SRAM tiles using the online softmax normaliser of Milakov and Gimelshein [13], avoiding the

TABLE I
Label distribution per split (annotations).

Type	Label	Train	Val	Test
Inclusion	included	88	35	27
	not included	13	3	7
	not enough information	109	27	38
	not applicable	5	1	3
Exclusion	excluded	8	2	0
	not excluded	291	101	91
	not enough information	65	26	28
	not applicable	23	11	12
	Patients / annotations	37/602	8/206	8/206

$O(T^2)$ score matrix. Speculative decoding [14], [15] lets a cheap draft model propose k tokens that the target verifies in one forward pass; under greedy acceptance the output is provably identical to target-only greedy decoding. Limitations: speed-ups require high acceptance and a draft that is much cheaper than the target, which is not guaranteed for short structured outputs (§V-F).

III. Materials and Methods

A. Dataset, split and leakage control

The dataset has 1,015 criterion annotations for 53 patients and 103 trials. We drop gpt4_explanation, gpt4_sentences, gpt4_eligibility, explanation_correctness and the dataset's own training flag immediately after loading (an assertion guards against regressions). Following the protocol, unique patient IDs are sorted, shuffled with numpy.random.default_rng(42), and assigned 37/8/8 to train/validation/test. One training annotation with empty criterion text was removed after splitting (recorded in the manifest), giving 602/206/206 annotations. Table I shows the label distribution: it is highly imbalanced (not excluded dominates exclusion criteria) and the test split contains no excluded example, so that class's test recall is undefined; we report it on validation instead. 60% of annotations have no gold evidence sentence. The manifest (splits/manifest.json) lists patient IDs and per-patient counts, and unit tests verify the protocol and disjointness.

B. Input/output format and truncation

The prompt (Qwen chat format) contains a system instruction, the patient note as [id] sentence lines using the dataset's original sentence IDs, the criterion type, the criterion text, and the allowed labels for that type. The target is {"evidence_ids":[...],"label":"..."}<|im_end|>. Placing evidence before the label conditions the decision on the cited sentences. The prompt is tokenised as a patient prefix and a criterion suffix; the same function is used in training, evaluation and the application, which makes prefix-KV reuse across criteria exact. If a prompt exceeded 1,800 tokens, patient sentences would be dropped from the end and logged with a flag if gold evidence was lost; with the Qwen tokenizer the longest prompt is 610 tokens (mean 326), so no example was truncated (audit in the manifest). Explanations are generated deterministically from the predicted label and the quoted cited sentences; this guarantees that an explanation cannot introduce facts absent from the record and avoids training on GPT-4 explanations. Outputs are parsed strictly as JSON; a fallback searches for an allowed label string; otherwise the prediction is invalid and counted as wrong.

TABLE II
Open-weight decoder candidates.

	Qwen2.5-1.5B-I	Qwen2.5-0.5B-I	Llama-3.2-1B-I
Params	1.54B	0.49B	1.24B
Layers / hidden	28 / 1536	24 / 896	16 / 2048
Q / KV heads	12 / 2	14 / 2	32 / 8
Context	32,768	32,768	131,072
Vocabulary	151,936	151,936	128,256
Licence	Apache-2.0	Apache-2.0	Llama 3.2 Comm.
fp16 weights	3.1 GB	1.0 GB	2.5 GB
KV / token (fp16)	28 KB	12 KB	32 KB
Role	target	draft	rejected

C. Model selection

Table II compares the candidates. We select Qwen2.5-1.5B-Instruct: Apache-2.0 licence (no gating), GQA with only 2 KV heads (28 KB of fp16 KV cache per token, 6× less than multi-head attention), a 151,936-token vocabulary shared with the 0.5B sibling that serves as a speculative draft, and fp32 weights (6.2 GB) that fit a free 16 GB T4 for LoRA training without quantisation. Llama-3.2-1B has a more restrictive community licence and a gated download, and Llama-3.2-3B leaves little memory for training on a T4.

D. Fine-tuning

Objective: token-level cross-entropy on the answer tokens only (prompt tokens masked with −100). LoRA (r=16, α=32, dropout 0.05) is applied to all attention and MLP projections (18.46M trainable parameters, 1.18% of 1.56B). AdamW, learning rate 2×10^{-4}, cosine schedule with 5% warm-up, effective batch 16 (micro-batch 2 × 8 accumulation steps, gradient checkpointing), 3 epochs, fp32 base weights with fp16 autocast, seed 42, on one Kaggle T4 (759 s, peak 9.5 GB). A checkpoint is saved every epoch; the final model is the checkpoint with the best validation macro-F1 (tie-break: evidence micro-F1). The test split is only accessible with an explicit --final_test flag and was evaluated once.

E. System architecture

Fig. 1 shows the pipeline. The LoRA adapter is merged into the base weights ($W \leftarrow W + \frac{\alpha}{r}BA$) so that all inference code sees a dense model.

F. Cache-aware decoding (Task 3)

Our engine re-implements the Qwen2 forward pass (embedding, RMSNorm, RoPE, GQA, SwiGLU, tied LM head) using the model's weights; its logits match Hugging

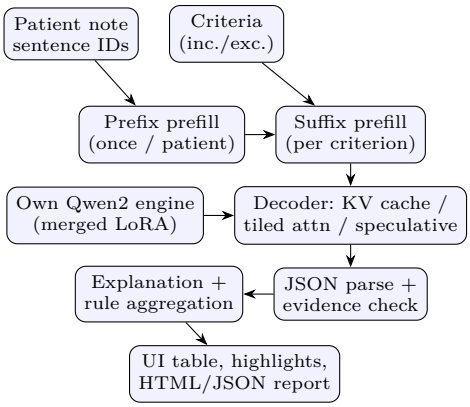


Fig. 1. TrialMatch-LLM inference workflow. The patient prefix KV cache is computed once and reused for every criterion.

Algorithm 1 Tiled attention with online softmax

for each query block Q_i do
  $m \leftarrow -\infty$, $\ell \leftarrow 0$, $O \leftarrow 0$
  for each key block (K_j, V_j) not entirely in the future do
    $S \leftarrow Q_i K_j^\top / \sqrt{d}$; mask future positions with $-\infty$
    $m' \leftarrow \max(m, \mathrm{rowmax}\, S)$; $\alpha \leftarrow e^{m-m'}$; $P \leftarrow e^{S-m'}$
    $\ell \leftarrow \alpha\ell + \mathrm{rowsum}\, P$; $O \leftarrow \alpha O + PV_j$; $m \leftarrow m'$
  end for
  output block i: O/ℓ
end for

Face within 10^{-4} (fp32 unit test). The cache stores, per layer, $K, V \in \mathbb{R}^{B \times h_{kv} \times C \times d}$ with a valid length L. Positional handling: a token at absolute position p is rotated by $p\,\theta_i$ before its key is written, so cached keys are never re-rotated; a step feeding token x_L only needs position L. Masking: prefill uses a causal mask; when T_q new queries attend to $L + T_q$ keys the mask is bottom-right aligned (query i sees keys $j \le L+i$). For single-token decoding all cached keys are visible and no mask is needed. Note that PyTorch's is_causal=True is top-left aligned and would be wrong for $T_q < T_k$, so we pass explicit masks. Growth: buffers are pre-allocated and doubled when full (amortised $O(1)$ per token); memory is $2\,N_{\text{layers}}\,h_{kv}\,d\,b$ bytes per token (b bytes per element). Rollback: cropping only resets L. The reference path recomputes the whole sequence every step. We also include a loop that passes an explicitly constructed DynamicCache, positions and cache_position to the HF model, to separate the effect of our engine from the effect of caching.

G. Tiled online-softmax attention (Task 4)

Algorithm 1 processes query blocks of size B_q and streams key/value blocks of size B_k, keeping a running row maximum m, normaliser ℓ and un-normalised output O. Blocks entirely in the causal future are skipped, and only blocks that straddle the diagonal are masked. Extra memory is $O(B_q B_k)$ instead of $O(T^2)$.

Screenshot: figures/app_screening.png

Fig. 2. Screening tab: criterion-level predictions, cited evidence highlighted in the note, preliminary rule-based outcome and report export.

H. Greedy speculative decoding (Task 5)

Invariant: with committed sequence $x_{1:n}$, the target cache holds $x_{1:n-1}$, and the draft cache holds a prefix of it. Each round (i) the draft greedily proposes $d_1..d_k$, feeding any committed tokens it has not cached; (ii) the target runs one forward pass over $[x_n, d_1..d_k]$ and returns $t_1..t_{k+1}$; (iii) the longest prefix with $d_i = t_i$ (a tokens) is accepted and t_{a+1} is appended (the correction on rejection, or a bonus token if $a{=}k$); (iv) both caches are cropped to $n + a$ entries, discarding the K/V of rejected tokens. Each round therefore emits between 1 and $k{+}1$ tokens that are identical to target-only greedy decoding. The draft is Qwen2.5-0.5B-Instruct (identical tokenizer, verified programmatically); we compare it to a draft fine-tuned with LoRA on the same training split.

I. Application

A Gradio application (Figs. 2–3) is started with start.bat/start.sh. It accepts unseen free-text notes (automatically sentence-split) and criteria, validates inputs (empty or short notes, too many or too long criteria, context overflow warnings), lets the user choose the inference mode, and shows per-criterion labels, cited IDs, evidence text, explanations, and missing-information flags, with the cited sentences highlighted. The preliminary patient–trial outcome is an explicit rule: likely ineligible if any inclusion criterion is not included or any exclusion criterion is excluded; otherwise needs review if any criterion has insufficient information; otherwise likely eligible. It is not a ground-truth label. Reports are exported as HTML and JSON with provenance (model, adapter, decoding mode, device, input hash, timestamp). A benchmark tab runs all decoding paths live on the user's input (Fig. 3). On the laptop CPU, screening the unseen 6-sentence diabetes note of Fig. 2 against 7 criteria takes 50 s with the KV-cache engine. The run also shows a characteristic error: "HbA1c between 7.5% and 10%" is answered not enough information although the note states 8.9%, i.e. numeric range checking is not reliable, so the outcome is correctly routed to needs manual review.

IV. Experimental Setup and Metrics

Hardware. Training and GPU benchmarks: Kaggle, 1× NVIDIA T4 (16 GB, no bf16), Python 3.13, PyTorch 2.11 (CUDA 13), transformers 5.22.2, peft 0.21.2. Application:

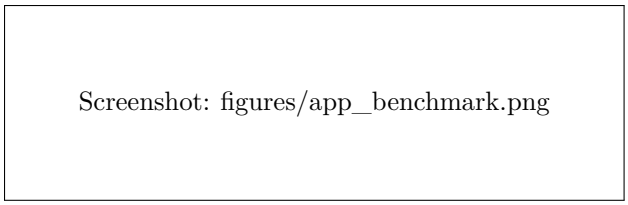


Fig. 3. Live inference benchmark tab: no-cache, KV-cache, tiled-attention and speculative paths on the same input, with token-agreement check.

Intel Core i5-10210U laptop CPU, 24 GB RAM, fp32. Decoding. Greedy, max_new_tokens=64, batch 8 with left padding for metric evaluation; identical prompts and rows before and after fine-tuning. Classification. Accuracy, macro-F1 over the gold classes present, per-class P/R/F1 and confusion matrices for inclusion, exclusion, and overall (8 type-specific classes). Evidence. Micro P/R/F1 sums TP/FP/FN over rows; macro averages row-level scores. Empty-set conventions: gold = $\emptyset$ and prediction = $\emptyset$ scores 1; exactly one empty scores 0. Because 60% of rows have empty gold evidence, we also report metrics restricted to rows with non-empty gold evidence (no convention needed) and the counts of the four empty/non-empty patterns. Timing. One warm-up run, then 3 repeats with torch.cuda.synchronize() around each phase; median reported. Peak memory from torch.cuda.max_memory_allocated. EOS is disabled for fixed-length KV benchmarks.

V. Results and Discussion

A. Fine-tuning and before/after comparison

Training took 759 s on one T4 with a peak of 9.5 GB (114 optimiser steps). Fig. 4 shows that the answer-token loss drops from 0.30 to about 0.12 during the first epoch and to about 0.06 by the end of epoch 3; the validation loss is lowest after epoch 1 (0.120) and rises slightly afterwards (0.126, 0.127), a mild sign of over-fitting. Validation macro-F1, however, keeps improving (0.263, 0.286, 0.353 after epochs 1–3, results/selection.json), because the extra epochs mainly improve the rare classes, which contribute little to the token loss but much to macro-F1. We therefore select by the task metric, not the loss, and the epoch-3 checkpoint is used everywhere.

Table III compares the same model before and after fine-tuning on identical rows with identical greedy decoding. Both versions always produce parseable JSON (strict-JSON rate 1.00), so the differences are in content. The base model systematically misreads the exclusion label semantics: it answers excluded for all 131 exclusion criteria on test (139/140 on validation), giving 0.0 exclusion accuracy, i.e. it interprets "excluded" as "this criterion is relevant" rather than "the patient meets an exclusion condition". On inclusion criteria it is already reasonable (0.627 accuracy). Fine-tuning fixes the exclusion semantics (0.687 accuracy), modestly improves inclusion (0.707), and

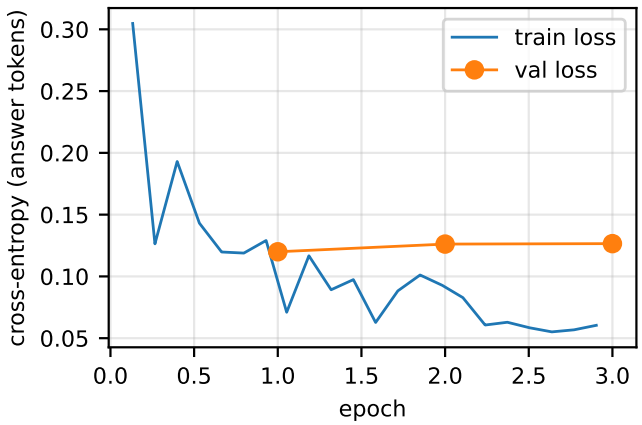


Fig. 4. LoRA training loss (per 5 steps) and validation loss (per epoch) on answer tokens.

triples evidence precision (micro-P 0.146→0.505) because the base model over-cites sentences.

B. Detailed classification and evidence analysis

Fig. 5 and Table IV give the per-class behaviour of the fine-tuned model on test. For inclusion criteria, the two frequent classes are learnt well (included F1 0.76, not enough information F1 0.76), but the rare classes are never predicted: the 7 not included cases become included (4) or not enough information (3), and the 3 not applicable cases become not enough information. For exclusion criteria the model has collapsed towards the majority class: not excluded has recall 0.96, but all 28 not enough information cases are also predicted not excluded, and only 3 of 12 not applicable cases are recovered (precision 0.75). The test split contains no excluded example; the model predicts it 3 times (all false positives), and on validation it finds both excluded cases (recall 1.0, precision 0.25). Macro-F1 (0.386) is therefore much lower than accuracy (0.694): it is dominated by classes with 3–13 training examples.

Evidence. On all test rows, micro P/R/F1 is 0.505/0.596/0.546 and macro F1 is 0.790; the macro score is inflated by the 128 rows where both gold and prediction are empty (counted as 1 by our convention). Restricted to the 61 rows with non-empty gold evidence, micro-F1 is 0.651 (inclusion 0.709, exclusion 0.489). Compared with the base model, spurious citations on gold-empty rows fall from 36 to 17 and exact set matches rise from 52.9% to 74.8%. Evidence for exclusion criteria is the weak point (micro recall 0.344): the model rarely cites the demographic sentence that annotators use to justify not excluded (e.g. age).

C. Error analysis

The fine-tuned model makes 63 label errors on the 206 test annotations. The automatic taxonomy in scripts/error_analysis.py assigns them (non-exclusively) to: unsupported certainty E2 (38), applicability errors E5

TABLE III
Criterion classification and evidence retrieval (greedy, same rows). Ev. = evidence; > 0 = rows with non-empty gold evidence.

Split	Model	Acc.	Macro-F1	Inc. mF1	Exc. mF1	Ev. micro-F1	Ev. macro-F1	Ev. micro-F1 (> 0)	Strict JSON
val	base	0.141	0.103	0.198	0.007	0.188	0.464	0.311	1.00
val	fine-tuned	0.694	0.353	0.328	0.378	0.530	0.703	0.631	1.00
test	base	0.228	0.192	0.336	0.000	0.236	0.608	0.403	1.00
test	fine-tuned	0.694	0.386	0.379	0.395	0.546	0.790	0.651	1.00

TABLE IV
Per-class test metrics, fine-tuned model.

Type	Class	P	R	F1	n
Inc.	included	0.64	0.93	0.76	27
	not included	0.00	0.00	0.00	7
	not enough info.	0.78	0.74	0.76	38
	not applicable	0.00	0.00	0.00	3
Exc.	excluded	0.00	–	–	0
	not excluded	0.70	0.96	0.81	91
	not enough info.	0.00	0.00	0.00	28
	not applicable	0.75	0.25	0.38	12

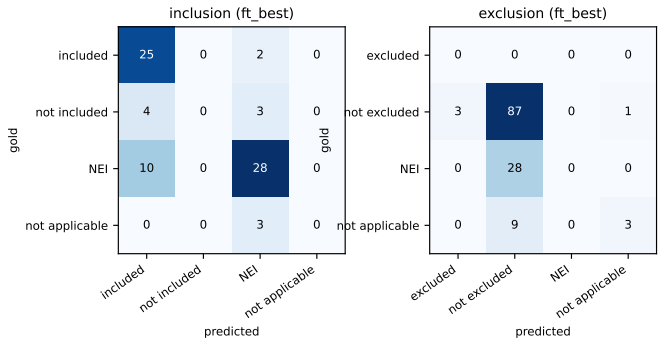


Fig. 5. Test confusion matrices of the fine-tuned model (NEI = not enough information).

(13), polarity errors E4 (7), missed available information E3 (5) and right-evidence/wrong-label contradictions E9 (3); no output was invalid (E1). Eight representative cases (full list in results/test/ft_best/error_analysis.md):

1) Missing information treated as negative (E2). Exclusion "estimated pre-operative GFR <30 mL/min": the note contains no renal data (gold not enough information), the model answers not excluded with no evidence. This is the dominant failure: it treats absence of a finding as proof that the exclusion does not apply.
2) Unsupported certainty with available context (E2). Exclusion "patients were moribund": annotators cite sentences 0–4 describing a critically ill patient but judge the information insufficient; the model answers not excluded without citing anything.
3) Missed available information (E3). Inclusion "diagnosed with shock, ARDS, or both": gold included with sentences 2–3 (pale, tachypnoeic, tachycardic, blood pressure 80/60); the model answers not enough information, failing to map haemodynamic findings to the diagnosis "shock".
4) Polarity error by topical similarity (E4). Inclusion "patients undergoing open abdominal aortic aneurysm repair": the patient has a pulsatile groin mass with lost distal pulses after femoral catheterisation; the model cites sentences 0–2 and answers included, matching the vascular topic instead of the specific procedure (gold not included). Two sibling criteria of the same trial fail identically.
5) Applicability (E5). Exclusion "pregnant women or women of childbearing age without adequate contraception" for a 48-year-old man (sentence 0): annotators label it not excluded citing the sex in sentence 0, the model answers not applicable without evidence. Both readings are defensible; the boundary between "does not apply" and "condition absent" is a labelling convention seen in only 28 training examples.
6) Applicability, reverse direction (E5). Exclusion "previous tonsillectomy" for a sleep-apnoea trial: gold not applicable, predicted not excluded.
7) Right label, missing evidence (E8). Exclusion "less than 18 years of age": correctly not excluded, but the age sentence 0 is not cited; 20 correct predictions have incomplete evidence.
8) Contradictory evidence and label (E9). Inclusion "immune deficiency and early bone-marrow failure in childhood": the model cites the correct sentence ("48-year-old …common variable immunodeficiency") but answers included, ignoring the qualifier "in childhood"; gold not enough information. A similar case is a compound sleep-disordered-breathing definition that is only partially supported by "nighttime snoring, pauses in breathing".

Traceability and absent evidence. Every prediction links to its cited sentence IDs, the quoted text and the raw model output. 115 test predictions are definite labels without any cited sentence; most are not excluded decisions that mirror the annotation convention (291 training rows are not excluded with empty evidence), but they are exactly the E2 risk above. The application therefore labels such outputs "unsupported, should be reviewed" in the explanation and never presents missing evidence as proof.

TABLE V
Greedy decoding with/without KV cache (T4, fp16, 256 new tokens, median of 3).

In	Path	Prefill	tok/s	Peak MB	=ref
128	HF, no cache (ref.)	41 ms	14.5	3178	–
128	HF + explicit cache	44 ms	26.5	2998	✓
128	own engine + cache	30 ms	37.1	2974	✓
512	HF, no cache (ref.)	134 ms	5.4	3401	–
512	own engine + cache	77 ms	36.6	3012	✓
1024	HF, no cache (ref.)	305 ms	2.6	3699	–
1024	HF + explicit cache	292 ms	26.3	3301	✓
1024	own engine, no cache	185 ms	4.7	3044	✓
1024	own engine + cache	170 ms	37.0	3060	✓

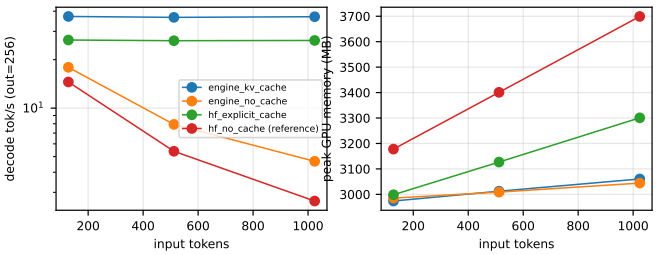


Fig. 6. Decode throughput and peak memory, cached vs. uncached.

D. KV cache

Table V and Fig. 6 report the fine-tuned (merged) model in fp16 on a T4. Correctness: all four paths produce identical greedy tokens in all nine (input, output) settings, i.e. 1,248 generated tokens per path and repeat with no mismatch, and outputs were identical across the three timed repeats. Speed: cached decoding runs at a constant 37 tok/s (own engine) and 26 tok/s (HF forward with our explicit cache) regardless of context length, whereas the uncached reference falls from 22.5 to 2.6 tok/s as the context grows, because every step re-processes the whole sequence ($O(T)$ work per token, $O(T^2)$ per sequence). At 1,024+256 tokens the engine is 14.2× faster than the uncached reference (6.9 s vs. 96.3 s decode). Our engine is also 1.4× faster than the HF cached loop and has 40% lower prefill latency at 1,024 tokens (170 vs. 290 ms); we attribute this to less Python-level overhead per layer (no mask construction or generic output objects), not to a different algorithm. Memory: the cache for 1,279 positions occupies 35 MB (28,672 B/token), negligible next to the 3.1 GB of weights; peak memory is lower with caching (3.06 vs. 3.70 GB) because the uncached path materialises activations for the full sequence at every step. Numerical differences: both paths compute the same function, but the cached step uses matrix–vector products and a different reduction order than the full-sequence matrix–matrix products, so fp16 logits are not bit-identical; an argmax flip can occur only at a near-tie, which we did not observe.

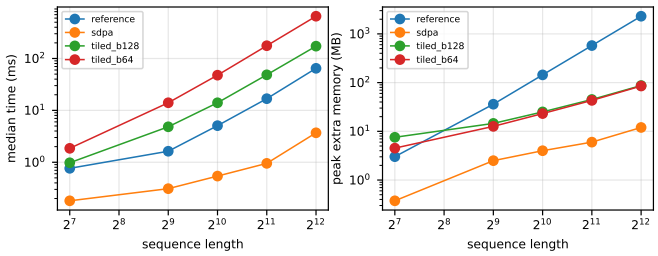


Fig. 7. Attention runtime and extra memory vs. sequence length (T4, fp16).

E. Tiled attention

Numerics. Over six shapes (including T_q=1 and T_q=7 queries against 300 cached keys with bottom-right masks), causal and non-causal masks and two tile sizes, the tiled kernel's maximum absolute error against an fp64 reference is 2.0×10^{-6} in fp32 and 9.8×10^{-4} in fp16, the same order as PyTorch SDPA (6.6×10^{-7} / 1.2×10^{-3}); the fp16 error is set by fp16 input rounding, not by the algorithm. A unit test with scores of magnitude 10^4 confirms that the running-maximum rescaling avoids overflow. Inside the full model, swapping our tiled kernel in reproduces Hugging Face logits within 10^{-4} (unit test), and the application exposes it as an inference mode.

Performance (Fig. 7, 1×12 heads, d=128, fp16, T4): at T=4096 the reference needs 64.7 ms and 2,304 MB of extra memory (it materialises fp32 score and probability matrices, $12\times4096^2\times4$ B = 805 MB each), our tiled kernel needs only 87 MB but takes 173 ms (B=128) or 652 ms (B=64), and fused SDPA (FlashAttention backend enabled) needs 3.7 ms and 12 MB. The educational kernel is therefore 47× slower than the fused kernel while achieving the same memory scaling. The reasons: each tile issues several separate GPU kernels launched from a Python loop ($O((T/B)^2)$ launches); every intermediate (S, P, rescaled O) is written to and read back from global HBM instead of staying in on-chip SRAM/registers; and the rescaling adds extra element-wise passes. A fused kernel runs the whole loop inside one CUDA kernel with tiles in shared memory. Larger tiles help (fewer launches) at the cost of more memory per tile.

F. Speculative decoding

We evaluate two workloads on 8 validation prompts: the real screening task (short JSON answers, EOS on) and a 128-token free-text summary (EOS off), for $k \in \{1, 2, 4, 6, 8\}$, with the fine-tuned target and either the base or a LoRA-fine-tuned 0.5B draft (Fig. 8, Table VI). Correctness: in all 160 (prompt, k, draft, workload) runs the output is token-identical to target-only greedy decoding. Acceptance: with the base draft, 56–82% of drafted tokens are accepted on the task; fine-tuning the draft on the same training split raises this to 84–96% (up to 6.6 tokens per target call at k=8), while acceptance

TABLE VI
Speculative decoding (task workload, 8 prompts; greedy target: 35 tok/s).

Draft	k	Accept.	Tok/call	Speed-up	=greedy
base 0.5B	1	0.83	1.80	0.77	✓
base 0.5B	4	0.56	2.87	0.62	✓
base 0.5B	6	0.72	4.11	0.81	✓
LoRA 0.5B	1	0.96	1.92	0.81	✓
LoRA 0.5B	4	0.84	4.15	0.82	✓
LoRA 0.5B	6	0.94	5.79	0.92	✓
LoRA 0.5B	8	0.87	6.60	0.87	✓

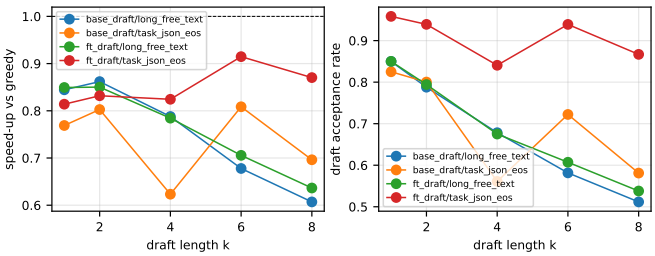


Fig. 8. Speculative decoding speed-up and acceptance vs. draft length k.

on free text is unchanged (51–85%), confirming that the gain comes from matching the task's output distribution. Speed: despite this, speculative decoding is slower than plain greedy in every configuration: 0.61–0.86× with the base draft and 0.64–0.92× with the fine-tuned draft (best: fine-tuned draft, k=6, task workload, 0.92×).

Why it is slower. One round costs $k\,c\,T_t + T_v$, where T_t is a target decode step, c the draft/target cost ratio, and $T_v \approx T_t$ the verification pass (at batch 1, processing k+1 tokens costs about as much as one). The speed-up is $\bar{a}/(1 + kc)$ with $\bar{a}$ tokens emitted per round. At batch 1 on a T4 both models are latency-bound (kernel launches and weight reads per layer), so cost scales with the number of layers rather than parameters: the 24-layer draft costs nearly as much per token as the 28-layer target ($c \approx 24/28 \approx 0.86$). Solving the measured fine-tuned-draft k=6 point ($\bar{a}$=5.79, speed-up 0.915) gives $c \approx 0.89$, consistent with this estimate, and the draft accounts for 38–84% of total time. A speed-up would need $c \ll 1$: a much shallower draft, a larger target (e.g. 7B), batching, or a CUDA-graph decode loop that removes per-layer launch overhead.

G. Design decisions, alternatives and failures

- LoRA vs. QLoRA vs. full fine-tuning. Full fine-tuning would need fp32 weights, gradients and two Adam moments (≈25 GB) and would over-fit 602 examples. LoRA with fp32 frozen weights peaked at 9.5 GB on the T4, so QLoRA's 4-bit quantisation (and its quantisation error and bitsandbytes dependency) was unnecessary; it remains available via --qlora.
- Model selection by task metric, not loss. Validation loss favoured epoch 1, validation macro-F1 favoured epoch 3 (0.263 vs. 0.353); since the deliverable is a classifier we used macro-F1.
- Evidence-first JSON. Chosen so that the label is generated conditioned on the cited sentences; a label-first ablation was not run within the time budget and is future work.
- Template explanations vs. generated rationales. Generated rationales would need GPT-4 explanations as targets (not permitted) and could hallucinate; templates quoting cited sentences are faithful by construction but shallow.
- Prefix caching. Placing the patient note before the criterion lets the application prefill each patient once: on the test set this removes 71% of prefill tokens (63,677→18,507 for 12.9 criteria per patient–trial pair on average).
- Fine-tuned vs. base draft. Raised acceptance by 13–29 points on the task workload but could not overcome the layer-count cost ratio.
- Failures observed and fixed. (i) The first training run crashed in peft because Kaggle ships an incompatible torchao; we disable the unused torchao path. (ii) The next run ran out of memory: on "T4×2" the HF Trainer silently wrapped the model in DataParallel, and without gradient checkpointing 28 layers of MLP activations for batch 4 exceed 16 GB; we pin one GPU, enable checkpointing and use micro-batch 2×8 accumulation (same effective batch). (iii) transformers v5 removed warmup_ratio. (iv) PyTorch's is_causal=True is top-left aligned and silently wrong for cached queries (T_q<T_k); we pass explicit bottom-right masks and a unit test checks the cached-query case against the reference. (v) One dataset row has empty criterion text and was removed.

VI. Limitations

Small data (602 training annotations, 37 patients) and severe class imbalance (8 training excluded examples, none in test) make per-class results for rare classes unreliable; 8 test patients give wide confidence intervals. Notes are short, synthetic-style SIGIR topics rather than full EHRs. The 1.5B model has limited medical knowledge. Explanations are faithful but shallow. CPU inference is slow (seconds per criterion). Speculative decoding gains are hardware-dependent. The aggregated outcome is a heuristic, not validated against trial enrolment. The system is an educational prototype and must not be used for clinical decisions.

VII. Conclusion

A 1.5B open-weight decoder fine-tuned with LoRA on 602 expert annotations reaches 0.694 accuracy and 0.386 macro-F1 on held-out patients (from 0.228 and 0.192), with evidence micro-F1 of 0.546, entirely on free hardware.

Its remaining errors are concentrated in rare labels and in treating missing information as negative, which the application surfaces rather than hides. On the systems side, an explicit KV cache gives exact outputs and a 14× decode speed-up at 1k context; a tiled online-softmax kernel is numerically exact but 47× slower than a fused kernel, illustrating that FlashAttention's benefit comes from kernel fusion and on-chip memory rather than the algorithm alone; and speculative decoding is exact and reaches 96% acceptance with a task-tuned draft but still slows decoding, because at batch 1 the cost of a sibling draft is governed by its depth. Future work: class-balanced training, label-first/evidence-first ablations, a shallower draft, and evaluation on full EHR notes.

Appendix A
AI-use statement

An AI coding assistant (Anthropic Claude) was used to draft code scaffolding, unit tests and parts of this report's text. All code was reviewed, executed and tested by the author (41 unit tests); all numbers come from logged runs in the repository. Design decisions, analysis and verification are the author's own responsibility.

Appendix B
Reproducibility

Code: https://github.com/Ali-Mazhar05/trialmatch-llm. Adapters: https://huggingface.co/AliMaz30/trialmatch-qwen2.5-1.5b-lora (target) and https://huggingface.co/AliMaz30/trialmatch-qwen2.5-0.5b-draft-lora (draft). Split manifest: splits/manifest.json. Logs: results/. Demo video: [URL].

References

[1] Q. Jin, Z. Wang, C. S. Floudas, F. Chen, C. Gong, D. Bracken-Clarke, E. Xue, Y. Yang, J. Sun, and Z. Lu, "Matching patients to clinical trials with large language models," Nature Communications, vol. 15, p. 9074, 2024.

[2] B. Koopman and G. Zuccon, "A test collection for matching patients to clinical trials," in Proc. 39th Int. ACM SIGIR Conf. Research and Development in Information Retrieval, 2016, pp. 669–672.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[4] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," Neurocomputing, vol. 568, 2024.

[5] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023.

[6] Qwen Team, "Qwen2.5 technical report," arXiv preprint arXiv:2412.15115, 2024.

[7] Llama Team, AI @ Meta, "The Llama 3 herd of models," arXiv preprint arXiv:2407.21783, 2024.

[8] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in International Conference on Learning Representations (ICLR), 2022.

[9] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[10] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently scaling transformer inference," in Proc. Machine Learning and Systems (MLSys), 2023.

[11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in Proc. 29th ACM Symposium on Operating Systems Principles (SOSP), 2023.

[12] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in Advances in Neural Information Processing Systems (NeurIPS), 2022.

[13] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," arXiv preprint arXiv:1805.02867, 2018.

[14] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in Proc. 40th International Conference on Machine Learning (ICML), 2023.

[15] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," arXiv preprint arXiv:2302.01318, 2023.